Database Management System

BCS-501 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** write each 7-mark answer in this order: definition, diagram, numbered points, small table, advantages and disadvantages, conclusion. **Bold words are the keywords examiners look for.** Always draw the diagram.

UNIT 1: ARCHITECTURE & ER MODELLING

Section A: 2-Mark Questions

Q1. What is a Data Dictionary? *[2 Marks]*

Ans. A **data dictionary** is a central store of **metadata** ("data about data"). It records table names, attributes, data types, constraints, indexes, users and access rights. An **active** dictionary is updated automatically by the DBMS. Example: `INFORMATION_SCHEMA` in MySQL.

Q2. Differentiate between Schema and Instance. *[2 Marks]*

Ans.

Schema	Instance
Logical design (structure) of the database	**Actual data** at a particular moment
Defined once with DDL; rarely changes	Changes with every insert, update or delete
Example: STUDENT(Roll, Name, Age)	Example: (101, 'Harsh', 21)

Q3. State the responsibilities of a DBA. *[2 Marks]*

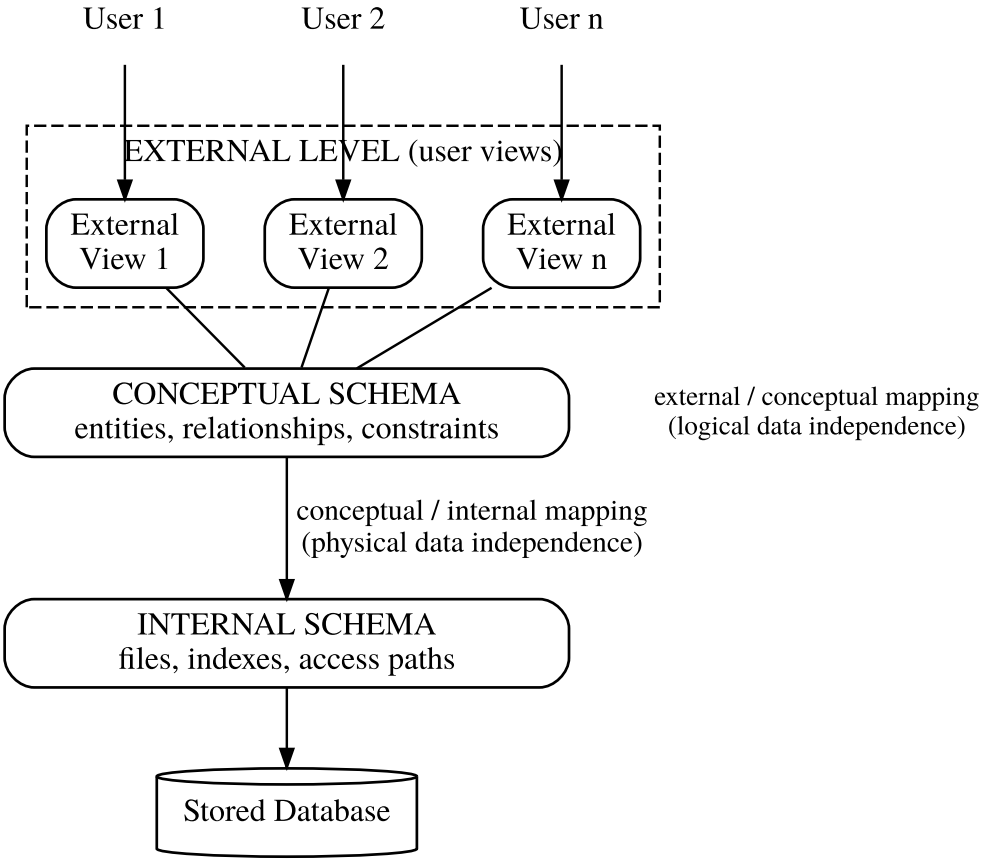
Ans. The **Database Administrator** is responsible for:

1. **Schema definition** (with DDL).
2. **Storage structure** and access methods (indexes).
3. **Granting authorization** (GRANT / REVOKE).
4. **Backup and recovery**.
5. **Integrity constraints**.
6. **Performance tuning**.

Section B: 7-Mark Questions

Q4. Explain the three-schema architecture and data independence. *[7 Marks]*

Ans. The **three-schema (ANSI/SPARC) architecture** divides a database into **three levels of abstraction**: **External, Conceptual and Internal**. Its main aim is **data independence**, which means the user's view is separated from physical storage.



Three-schema (ANSI/SPARC) architecture

Levels

1. **Internal level: how** data is stored, including files, indexes and access paths. Example: STUDENT stored in a B+ tree on Roll_No.
2. **Conceptual level: what** data is stored for the whole database: entities, relationships and constraints. It is designed by the **DBA**.
3. **External level:** the **user views**. Each user group sees only the part it needs. Example: Accounts sees (Roll, Fee_Status).

Data independence

- **Physical data independence:** the internal schema can change **without changing the conceptual schema**. Example: adding an index or changing disks.
- **Logical data independence:** the conceptual schema can change **without changing the external views**. Example: adding a column. This is **harder** to achieve.

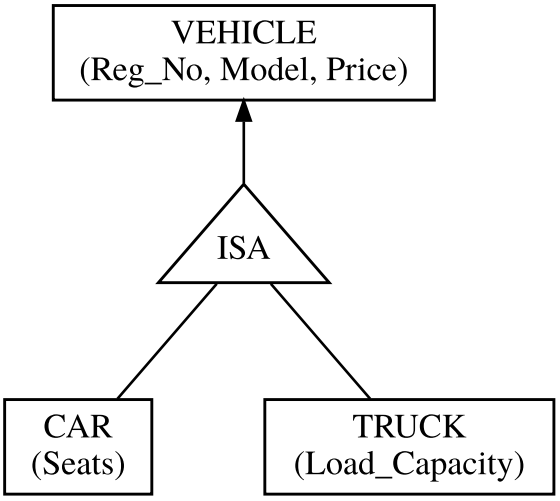
Physical DI	Logical DI
Change at the internal level	Change at the conceptual level
Conceptual/internal mapping updated	External/conceptual mapping updated
Easy	**Difficult**

Advantages: data independence, security through views, easier maintenance. **Disadvantages:** mapping overhead, complex design.

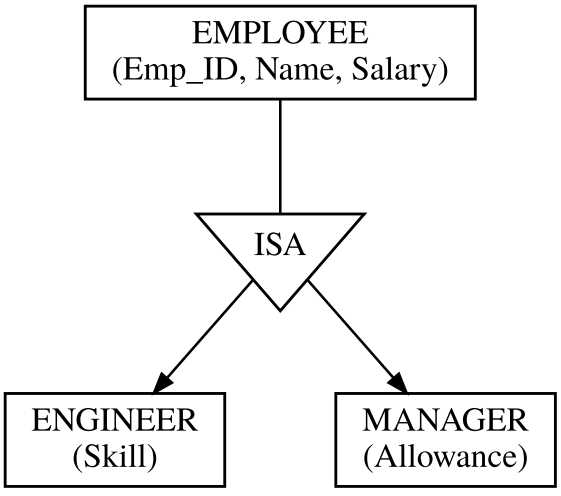
Conclusion: every RDBMS (Oracle, MySQL) follows this model: **views** are the external level and **tablespaces/indexes** are the internal level.

Q5. Differentiate Generalization, Specialization and Aggregation with examples. *[7 Marks]*

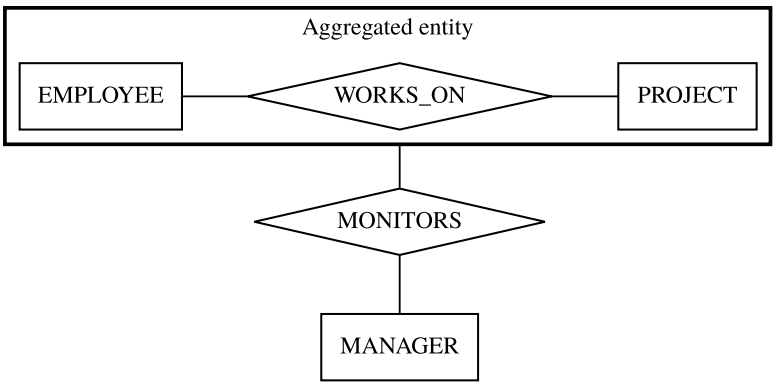
Ans. These are **Enhanced ER (EER)** concepts. Generalization and specialization use the **IS-A** relationship (inheritance). **Aggregation** treats a relationship as an entity.



Generalization (bottom-up): common attributes move up to VEHICLE



Specialization (top-down): EMPLOYEE split into sub-classes



Aggregation: the relationship (EMPLOYEE works on PROJECT) is treated as one entity

1. **Generalization (bottom-up):** lower-level entities with common attributes are combined into a **super-class**. Example: CAR + TRUCK → VEHICLE.
2. **Specialization (top-down):** an entity is split into **sub-classes** that inherit its attributes. Example: EMPLOYEE → ENGINEER, MANAGER.
3. **Aggregation:** a **relationship set is treated as a higher-level entity**, so it can take part in another relationship. Example: (EMPLOYEE works on PROJECT) is **monitored by** MANAGER.
4. **Constraints: disjoint / overlapping**, and **total / partial** participation.

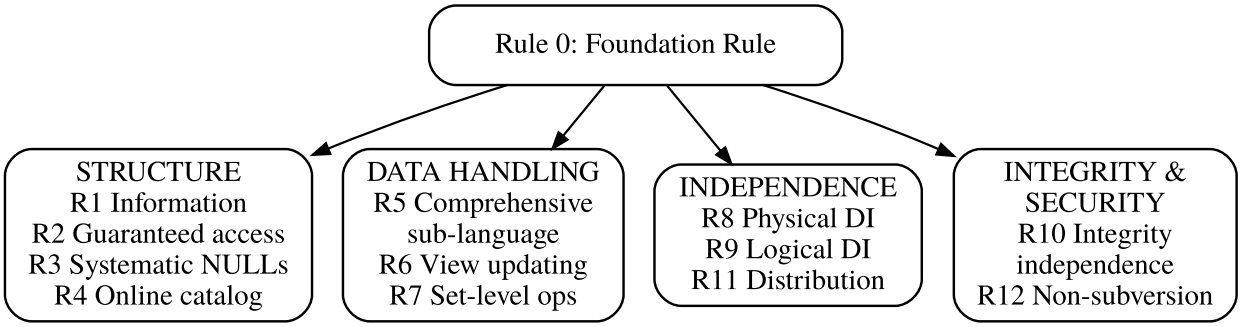
Basis	Generalization	Specialization	Aggregation
Approach	Bottom-up	Top-down	Abstraction of a relationship
Result	Fewer entities (super-class)	More entities (sub-classes)	Relationship becomes an entity
Relation type	IS-A	IS-A	HAS-A

Conclusion: used to model **ERP, banking and university** databases with entity hierarchies.

Q6. Explain Codd's 12 rules.

[7 Marks]

Ans. Dr. E.F. Codd (1985) gave **13 rules (Rule 0 to Rule 12)** that a DBMS must follow to be **fully relational**.



Codd's rules grouped by purpose

Rule	Name	Meaning
0	Foundation	Manage data **only through relational** capabilities
1	Information	All data, including metadata, is stored **in tables**
2	Guaranteed access	Any value = **table + primary key + column**
3	NULL treatment	NULL handled uniformly as missing data
4	Online catalog	Metadata stored as tables and queried with SQL
5	Data sub-language	One complete language (**SQL**) for DDL, DML, security
6	View updating	Theoretically updatable views must be updatable
7	Set-level operations	Insert / update / delete on **sets of rows**
8	Physical DI	Storage changes do not affect applications
9	Logical DI	Table changes do not affect applications
10	Integrity independence	Constraints stored in the catalog, not in programs
11	Distribution independence	Users unaware of data distribution
12	Non-subversion	Low-level access cannot **bypass** the rules

> **Exam Tip:** no commercial RDBMS satisfies all 12 rules fully. Most satisfy 8 to 10.

Conclusion: Codd's rules are the **benchmark** for judging how relational a DBMS is.

UNIT 2: RELATIONAL ALGEBRA & CALCULUS

Section A: 2-Mark Questions

Q1. Define Cartesian product and Theta join. *[2 Marks]*

Ans.

- **Cartesian product (R × S):** combines **every tuple of R with every tuple of S**. If R has m tuples and S has n, the result has **m × n tuples**.
- **Theta join:** $R \bowtie_\theta S = \sigma_\theta (R \times S)$, a join on any condition θ (<, >, =, ≠). With only '=', it is an **equi-join**.

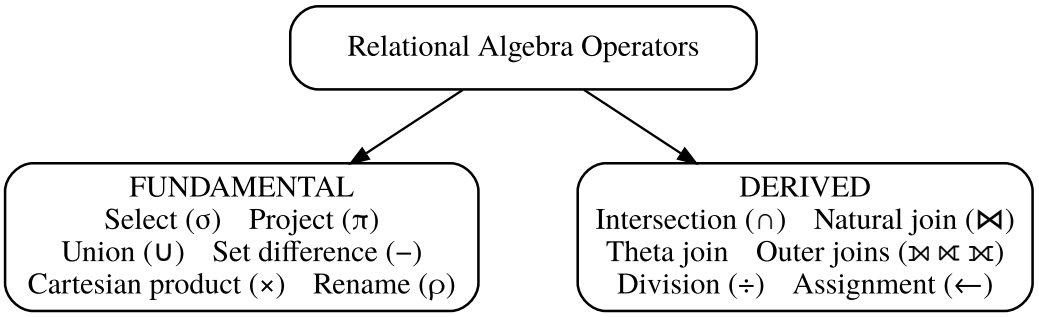
Q2. How do foreign keys cause cascading deletes? *[2 Marks]*

Ans. If a foreign key is declared with **ON DELETE CASCADE**, deleting a parent row **automatically deletes all the child rows** that refer to it. This keeps referential integrity, with no orphan rows. `FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID) ON DELETE CASCADE`

Section B: 7-Mark Questions

Q3. Explain relational algebra operators: natural join, outer joins and division. *[7 Marks]*

Ans. Relational algebra is a **procedural** query language. Its operators take relations as input and give a relation as output.



Classification of relational algebra operators

Fundamental operators:

- Select σ (rows)
- Project π (columns)
- Union ∪
- Set difference −
- Cartesian product ×
- Rename ρ

Example tables: EMP(EID, Name, DID) = {(1, Amit, 10), (2, Riya, 20), (3, Karan, 40)}, DEPT(DID, DName) = {(10, CSE), (20, IT), (30, ECE)}

1. **Natural join (EMP ⋈ DEPT):** equality on **common attributes** (DID), with duplicate columns removed. Result: (1, Amit, 10, CSE), (2, Riya, 20, IT).
2. **Outer joins:** keep **unmatched tuples** and fill the gaps with **NULL**.
   - **Left (⟕):** also gives (3, Karan, 40, NULL).
   - **Right (⟖):** also gives (NULL, NULL, 30, ECE).
   - **Full (⟗):** both of the above.
3. **Division (R ÷ S):** used for **"for all"** queries. ENROLLED = {(A,C1), (A,C2), (B,C1), (C,C1), (C,C2), (C,C3)}, COURSE = {C1, C2}.

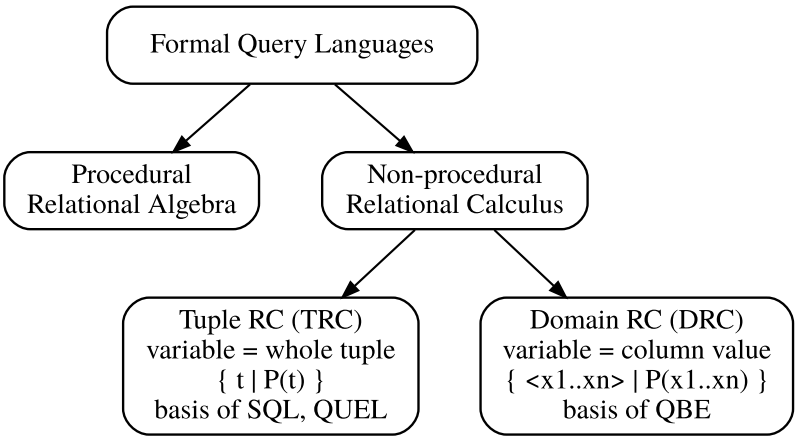
$$R \div S = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R) = \{A, B, C\} - \{B\} = \{A, C\}$$

Join	Unmatched tuples	Duplicate columns
Theta / Equi	Dropped	Kept
Natural	Dropped	Removed
Outer (L / R / Full)	**Kept with NULL**	Removed

Conclusion: relational algebra is used **inside the query optimizer** to build execution plans for SQL.

Q4. Differentiate between TRC and DRC with syntax. *[7 Marks]*

Ans. Relational calculus is **non-procedural**: the user says *what* is wanted, not *how*. In **TRC** variables are **tuples**. In **DRC** variables are **column values (domains)**.



Formal query languages

Example relation: STUDENT(Roll, Name, Age, Dept)

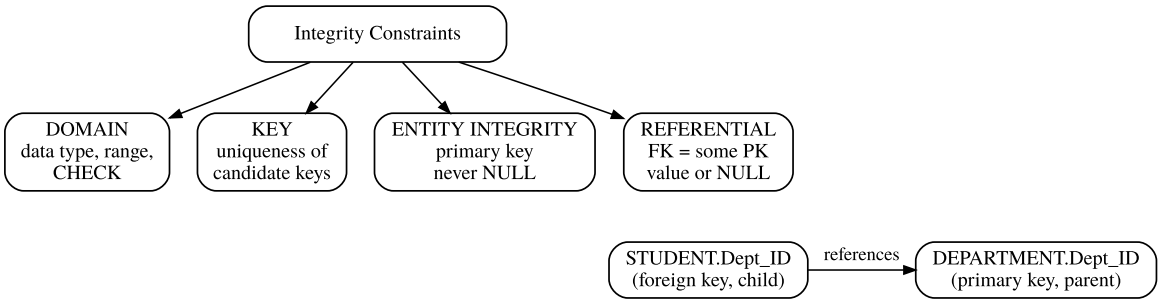
- **TRC:** names of students older than 20: $\{ t.\text{Name} \mid t \in \text{STUDENT} \wedge t.\text{Age} > 20 \}$
- **DRC:** the same query: $\{ \langle n \rangle \mid \exists r, a, d \ (\langle r, n, a, d \rangle \in \text{STUDENT} \wedge a > 20) \}$
- **Safety:** $\{t \mid \neg (t \in R)\}$ is **unsafe**, because it gives an infinite result. Safe TRC = safe DRC = relational algebra (**relational completeness**).

Basis	TRC	DRC
Variable	Whole **tuple**	Single **attribute value**
Syntax	{ t | P(t) }	$\{ \langle x_1 \ldots x_n \rangle \mid P(x_1 \ldots x_n) \}$
Attribute access	t.Name	positional ⟨r, n, a, d⟩
Language based on it	**SQL**, QUEL	**QBE**

Conclusion: TRC is the basis of SQL's SELECT-FROM-WHERE, and DRC is the basis of QBE (MS Access).

Q5. Explain domain, entity and referential integrity with examples. *[7 Marks]*

Ans. Integrity constraints are rules that keep data **valid and consistent**. They are enforced automatically by the DBMS.



Types of integrity constraints, and the foreign-key link

1. **Domain constraint:** every value must come from its **domain** (type, range). Example: `Age INT CHECK (Age BETWEEN 17 AND 30)`, so Age = 'abc' or 45 is **rejected**.
2. **Key constraint:** candidate key values must be **unique**.
3. **Entity integrity: a primary key can never be NULL**. Example: (NULL, 'Aman') is rejected.
4. **Referential integrity:** a **foreign key must match a primary key value, or be NULL**. Example: STUDENT.Dept_ID = 50 is rejected if 50 is not in DEPARTMENT.
5. **On violation:** RESTRICT, **CASCADE**, SET NULL or SET DEFAULT.

Constraint	Applies to	Rule	Violated by
Domain	Every attribute	Valid type / range	Insert, Update
Entity	Primary key	PK ≠ NULL	Insert, Update
Referential	Foreign key	FK = PK value or NULL	Insert (child), Delete (parent)

Conclusion: constraints prevent **orphan records and invalid data** in banking and ERP systems.

UNIT 3: DATABASE DESIGN & NORMALIZATION

Section A: 2-Mark Questions

Q1. Define insertion and update anomalies. *[2 Marks]*

Ans.

- **Insertion anomaly:** a fact cannot be inserted without unrelated data. Example: a new department cannot be added until a student joins it.
- **Update anomaly:** the same fact is stored in many rows, so a change must be repeated everywhere, otherwise the data becomes **inconsistent**.
- *(Deletion anomaly:* deleting one fact loses another.)

Q2. Trivial vs. non-trivial FD; Armstrong's axioms. *[2 Marks]*

Ans.

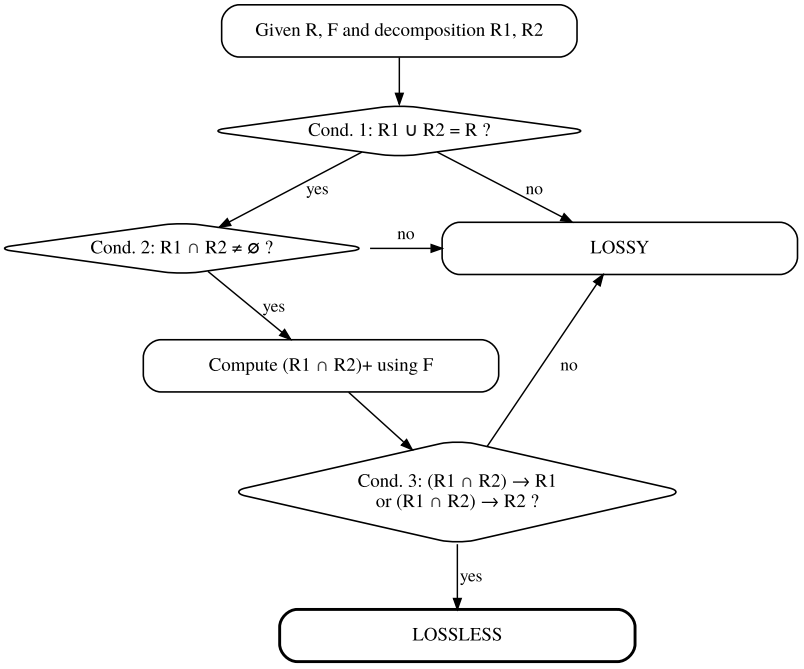
- **Trivial FD:** $X \rightarrow Y$ with $Y \subseteq X$ (for example {Roll, Name} → Name).
- **Non-trivial FD:** $Y \nsubseteq X$ (for example Roll → Name).

- **Armstrong's axioms:**
  - **Reflexivity:** if $Y \subseteq X$ then $X \to Y$.
  - **Augmentation:** if $X \to Y$ then $XZ \to YZ$.
  - **Transitivity:** if $X \to Y$ and $Y \to Z$ then $X \to Z$.

Section B: 7-Mark Questions

Q3. Prove whether a decomposition is lossless, and explain dependency preservation. *[7 Marks]*

Ans. A decomposition of R into R1 and R2 is **lossless** if $R_1 \bowtie R_2 = R$, that is, the join produces no **spurious tuples**.



Lossless-join test for a binary decomposition

Test: (1) $R_1 \cup R_2 = R$, (2) $R_1 \cap R_2 \neq \emptyset$, (3) $(R_1 \cap R_2) \to R_1$ or $(R_1 \cap R_2) \to R_2$.

Example: R(A, B, C, D, E), F = {A → BC, CD → E, B → D, E → A}, R1(A, B, C), R2(A, D, E)

1. $R_1 \cup R_2 = ABCDE = R$ ✓
2. $R_1 \cap R_2 = \{A\} \neq \emptyset$ ✓
3. A^+: A → BC gives ABC; B → D gives ABCD; CD → E gives **ABCDE**. So A → R1 ✓

Result: LOSSLESS.

Dependency preservation: the decomposition is preserving if $(F_1 \cup F_2)^+ = F^+$.

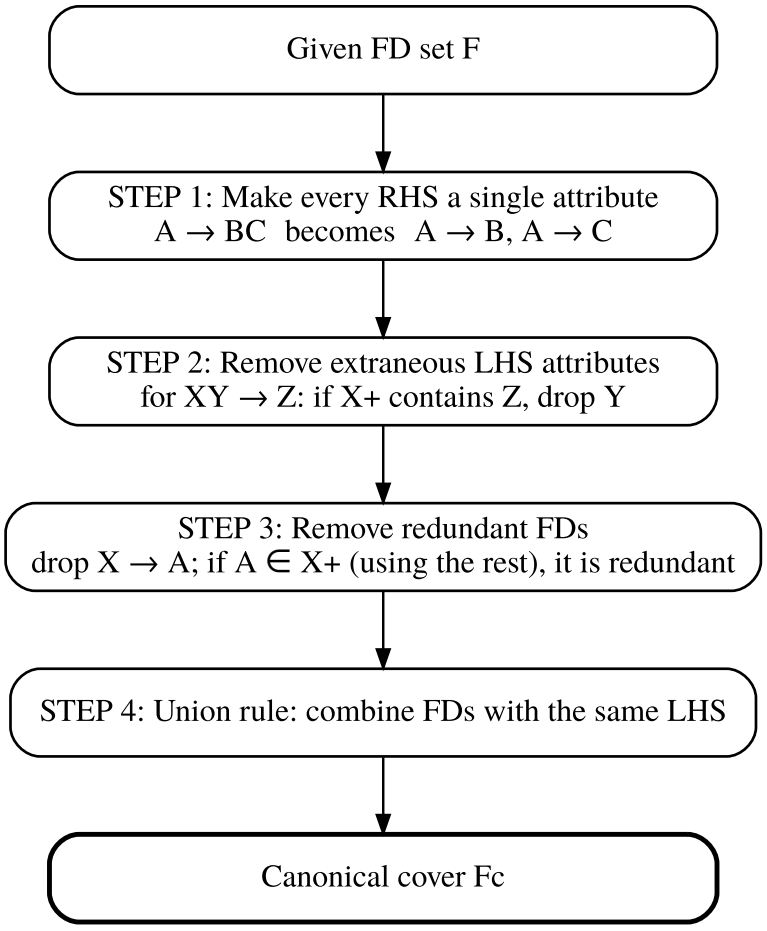
- F1 = {A → BC, BC → A} and F2 = {A → DE, E → A}.
- B → D: $B^+ = \{B\}$, which does not contain D, so it is **not preserved**.
- **Result: lossless but NOT dependency preserving.**

Lossy example: R(A, B, C) decomposed into R1(A, B) and R2(B, C), with only A → B. Then $B^+ = B$ is not a key of either part, so the join gives spurious tuples.

Conclusion: lossless join is **mandatory**. Dependency preservation is desirable (3NF always gives both; BCNF may lose dependencies).

Q4. Find the canonical cover of F = {A → B, C → B, D → ABC, AC → D}. *[7 Marks]*

Ans. A **canonical cover Fc** is the **minimal equivalent set** of FDs: single-attribute RHS, no extraneous attributes and no redundant FDs.



Algorithm for finding the canonical cover

Step 1: Split the RHS.

$$F = \{A \to B,\ C \to B,\ D \to A,\ D \to B,\ D \to C,\ AC \to D\}$$

Step 2: Extraneous LHS attributes in AC → D.

- $A^+ = AB$ does not contain D, so C is needed.
- $C^+ = CB$ does not contain D, so A is needed. **AC → D stays.**

Step 3: Remove redundant FDs.

FD	Closure without this FD	Redundant?
A → B	$A^+ = A$	No
C → B	$C^+ = C$	No
D → A	$D^+ = DBC$	No
D → B	$D^+ = DACB$ (via D → A, A → B)	**Yes, remove**
D → C	$D^+ = DAB$	No
AC → D	$AC^+ = ACB$	No

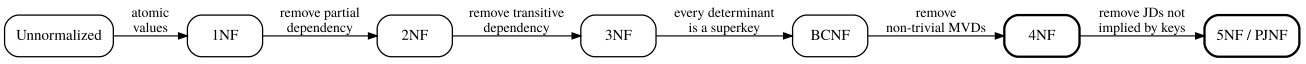
Step 4: Union. D → A and D → C combine to D → AC.

$$F_c = \{A \to B,\ C \to B,\ D \to AC,\ AC \to D\}$$

Conclusion: the canonical cover is the input to **3NF synthesis** and makes FD checking cheap.

Q5. Explain MVD and 4NF, and join dependency and 5NF. *[7 Marks]*

Ans. A **multivalued dependency** $X \twoheadrightarrow Y$ means that one X value has a **set of Y values independent of the other attributes**. **4NF** removes MVDs. A **join dependency** means the relation can be rebuilt **only by joining 3 or more projections**. **5NF** removes such JDs.



Normal forms: each step removes one kind of dependency

4NF example: STUDENT(SID, Course, Hobby). Courses and hobbies are independent, so SID →→ Course and SID →→ Hobby.

- S1 with 2 courses and 2 hobbies needs **4 rows**. This is redundant.
- **4NF rule:** for every non-trivial X →→ Y, **X must be a superkey**. SID is not, so the relation is not in 4NF.
- **Decompose** into (SID, Course) and (SID, Hobby).

5NF example: SPJ(Supplier, Part, Project) = {S1P1J2, S1P2J1, S2P1J1, S1P1J1}.

- SP ⋈ PJ gives a spurious tuple S2P1J2. Joining also with JS removes it.
- So $*\,(SP, PJ, JS)$ holds. **Decompose into 3 tables** (SP, PJ, JS) to reach 5NF.

Basis	4NF	5NF
Removes	MVD	Join dependency
Pre-condition	BCNF	4NF
Splits into	2 tables	3 or more tables

Conclusion: 4NF and 5NF remove the redundancy that BCNF cannot. They are used in supply-chain style schemas.

UNIT 4: TRANSACTIONS & CONCURRENCY CONTROL

Section A: 2-Mark Questions

Q1. Dirty read and unrepeatable read. *[2 Marks]*

Ans.

- **Dirty read (W-R conflict):** T2 reads data written by T1 **before T1 commits**. If T1 aborts, T2 has used an invalid value.
- **Unrepeatable read (R-W conflict):** T1 reads A twice and gets **different values**, because T2 updated and committed A in between.

Q2. Starvation vs. deadlock. *[2 Marks]*

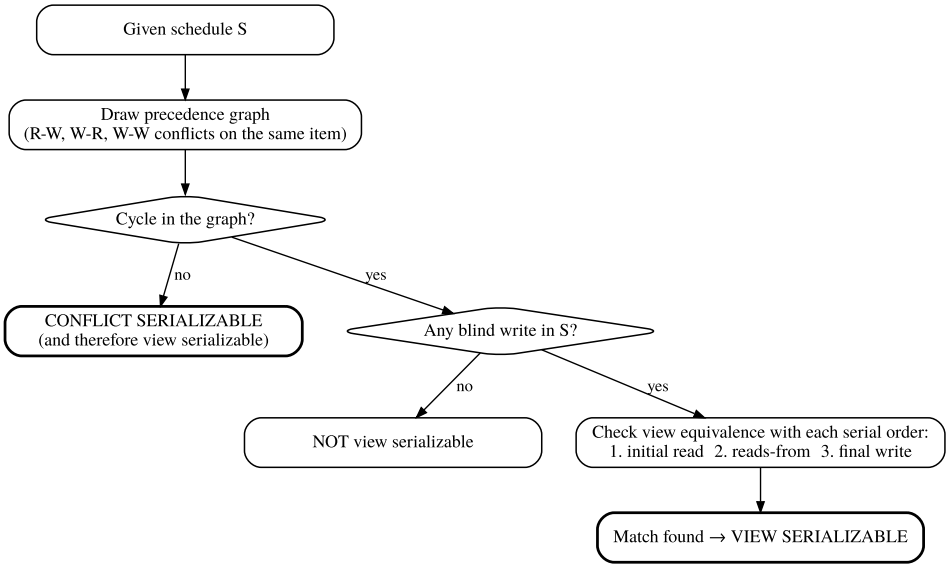
Ans.

Starvation	Deadlock
One transaction **waits indefinitely** while others proceed	Transactions **wait for each other in a cycle**, so none proceeds
Fix: aging, FIFO queue	Fix: wait-die / wound-wait, wait-for graph

Section B: 7-Mark Questions

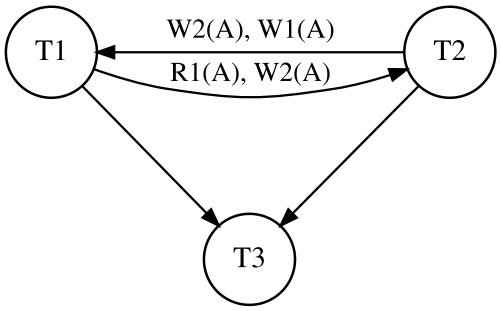
Q3. Conflict vs. view serializability; test a schedule. *[7 Marks]*

Ans. A schedule is **serializable** if it is equivalent to some **serial schedule**. **Conflict serializable:** it can be turned into a serial schedule by swapping non-conflicting operations. **View serializable:** it gives every transaction the same reads and the same final writes as a serial schedule.



Testing a schedule for conflict and view serializability

Schedule: R1(A), W2(A), W1(A), W3(A)



Precedence graph: the cycle T1 → T2 → T1 means S is NOT conflict serializable

Step 1: Conflict test. The edges T1 → T2 and T2 → T1 form a **cycle**, so S is **not conflict serializable**.

Step 2: View test against the serial order T1 → T2 → T3. W2 and W3 are **blind writes**.

Condition	S	T1 → T2 → T3
Initial read of A	T1	T1 ✓
Reads-from	none	none ✓
Final write of A	T3	T3 ✓

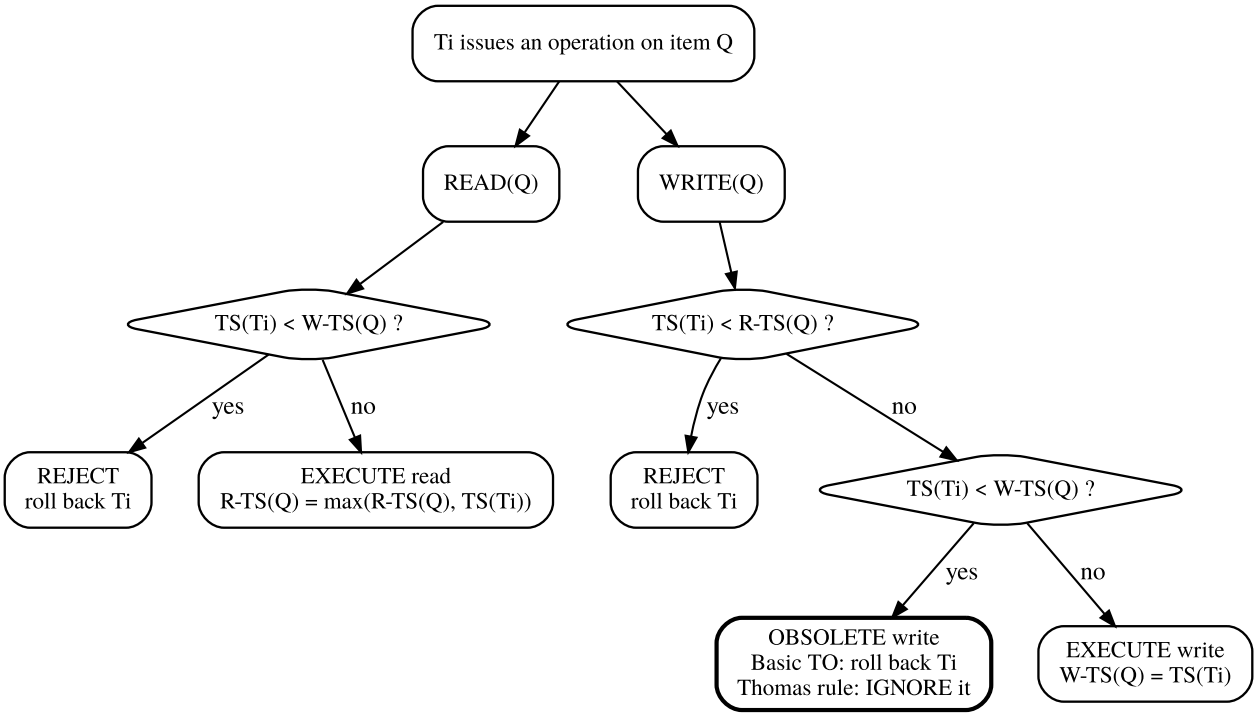
Result: VIEW serializable but not conflict serializable.

Conflict	View
Precedence graph test	Initial read, reads-from, final write
Polynomial	**NP-complete**
Subset (stricter)	Superset (allows blind writes)

Conclusion: in practice, DBMSs enforce **conflict serializability** (2PL, timestamps).

Q4. Explain the Timestamp Ordering protocol and the Thomas Write Rule. *[7 Marks]*

Ans. Timestamp Ordering (TO) is a **lock-free** protocol. Each transaction gets a unique timestamp TS(Ti), and conflicting operations run in **timestamp order**. Each item Q keeps **R-TS(Q)** and **W-TS(Q)**.



Basic timestamp ordering, with the Thomas Write Rule change

Rules:

1. **Read(Q):** if TS(Ti) < W-TS(Q), **roll back Ti**. Otherwise read, and set R-TS(Q) = max(R-TS(Q), TS(Ti)).
2. **Write(Q):** if TS(Ti) < R-TS(Q), **roll back Ti**.
3. **Write(Q):** if TS(Ti) < W-TS(Q), the write is **obsolete**. Basic TO **rolls back** Ti. The **Thomas Write Rule ignores the write** and lets Ti continue.
4. Otherwise write, and set W-TS(Q) = TS(Ti).

Example: TS(T1) = 10, TS(T2) = 20, with operations R1(Q), W2(Q), W1(Q).

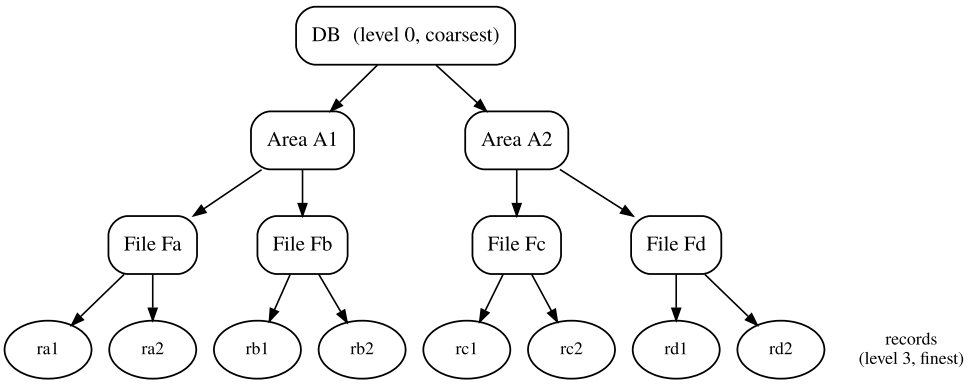
- At W1(Q): 10 < W-TS = 20, so the write is obsolete.
- **Basic TO:** roll back T1. **Thomas rule:** ignore W1(Q).

Advantages: deadlock-free, serializable. **Disadvantages:** starvation, cascading rollbacks, timestamp overhead.

Conclusion: the basis of **MVCC** in PostgreSQL and Oracle.

Q5. Explain multiple granularity locking and intention locks. *[7 Marks]*

Ans. Multiple granularity locking lets transactions lock items of **different sizes** (DB → Area → File → Record) arranged as a tree. Locking a node **implicitly locks its descendants**.



Granularity hierarchy: lock top-down, unlock bottom-up

Lock modes:

- **S** (shared) and **X** (exclusive).
- **IS / IX:** intention to take S / X locks **lower** in the tree.
- **SIX:** S on the whole subtree plus IX (read all, update a few).

Requested ↓ / Held →	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	×
IX	✓	✓	×	×	×
S	✓	×	✓	×	×
SIX	✓	×	×	×	×
X	×	×	×	×	×

Rules:

1. Lock the **root first**.
2. S or IS on a node requires IS or IX on its parent.
3. X, SIX or IX on a node requires IX or SIX on its parent.
4. Follow 2PL.
5. Unlock **bottom-up**.

- **Purpose of intention locks:** conflicts are checked only at the **ancestors**, so the whole subtree does not need to be searched.
- **Lock escalation:** many row locks (for example more than 5000) are replaced with **one table lock**, which saves memory but lowers concurrency.

Conclusion: used in **SQL Server, DB2 and MySQL InnoDB**.

UNIT 5: RECOVERY & FILE ORGANIZATION

Section A: 2-Mark Questions

Q1. What is Write-Ahead Logging (WAL)? *[2 Marks]*

Ans. WAL rule: the **log record of a change must reach stable storage before the changed data page is written to disk**, and all of a transaction's log records must be written before it commits. This makes UNDO and REDO possible after a crash.

Q2. Static vs. dynamic hashing; primary, clustering and secondary index. *[2 Marks]*

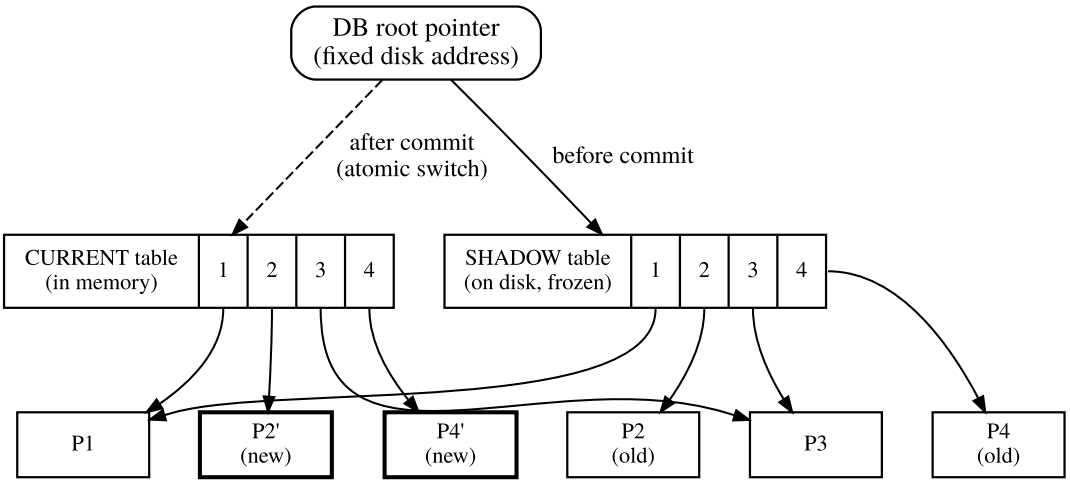
Ans.

- **Static hashing:** a **fixed** number of buckets, with overflow chains.
- **Dynamic hashing:** buckets **grow and shrink** (extendible hashing).
- **Primary index:** on the ordering **key** field (sparse).
- **Clustering index:** on an ordering **non-key** field.
- **Secondary index:** on a **non-ordering** field (dense; there can be many).

Section B: 7-Mark Questions

Q3. Explain Shadow Paging with its advantages and disadvantages. *[7 Marks]*

Ans. Shadow paging is a recovery technique that **needs no log** (for a single user). It keeps two page tables: the **current page table** and the **shadow page table**. The shadow table is never changed during the transaction.



Shadow paging: the shadow table still points to the old pages; pages 2 and 4 were copied on write

Working:

1. At the start, **copy** the current page table to make the shadow page table.
2. **On write (copy-on-write):** copy the page to a **free block**, update the copy, and point the **current table** to it.

3. **Commit:** flush the pages and the current table, then **atomically switch the DB root pointer** to the current table.
4. **Crash or abort:** the root still points to the **shadow table**, which is the old consistent state. **No UNDO or REDO** is needed.

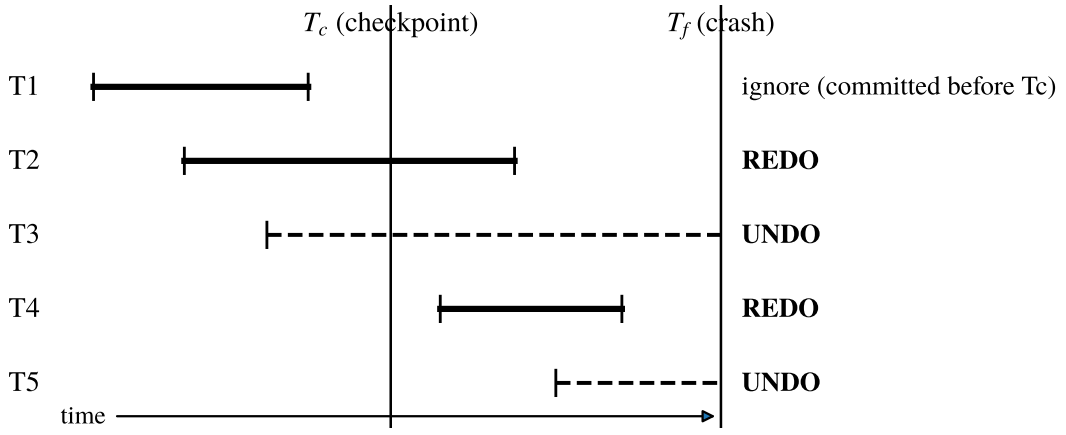
Shadow Paging	Log-based Recovery
No log	Needs a log
Recovery is very fast	Needs UNDO / REDO
High commit overhead	Low commit overhead
Hard with concurrency	Easy with concurrency

Advantages: no log overhead, fast recovery, simple atomic commit. **Disadvantages:** costly page-table copying, **data fragmentation**, garbage collection needed.

Conclusion: used in **System R** and in copy-on-write file systems (ZFS).

Q4. Explain checkpointing; deferred vs. immediate modification. *[7 Marks]*

Ans. Log-based recovery records every change: ⟨Ti start⟩, ⟨Ti, X, old, new⟩ and ⟨Ti commit⟩. A **checkpoint** limits how much of the log must be scanned after a crash.



Checkpoint recovery: transactions are classified by when they started and committed relative to the checkpoint Tc and the crash Tf

Checkpoint steps:

1. Write the log records in memory to stable storage.
2. Write the modified buffer blocks to disk.
3. Write a ⟨checkpoint L⟩ record.

On recovery, scan back only to the last checkpoint: **committed transactions are REDONE, uncommitted ones are UNDONE**, and transactions committed before Tc are ignored.

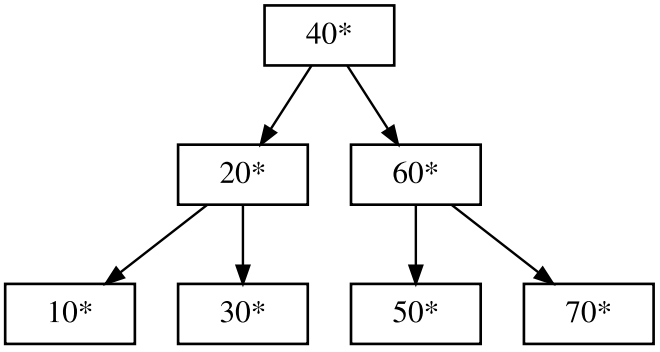
Example: T0: A = 1000 → 950, B = 2000 → 2050 (committed); T1: C = 700 → 600 (not committed).

Basis	Deferred modification	Immediate modification
DB updated	Only **after commit**	Before or after commit
Log record	⟨Ti, X, new⟩	⟨Ti, X, **old, new**⟩
Recovery	**REDO only**	**UNDO + REDO**
In the example	REDO(T0), ignore T1	REDO(T0), UNDO(T1) so C = 700

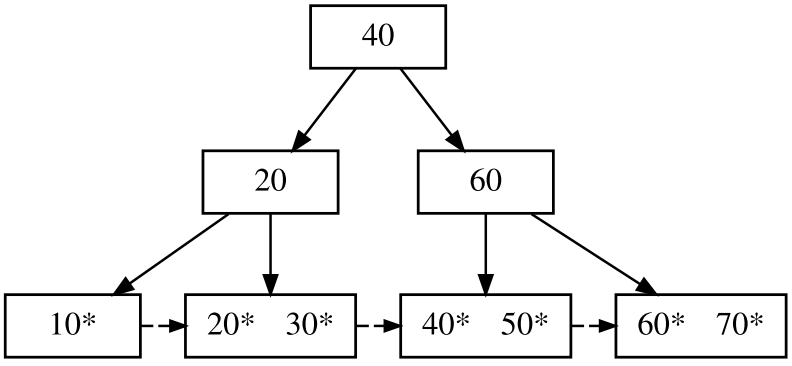
Conclusion: immediate modification with checkpoints is the basis of **ARIES** (DB2, SQL Server).

Q5. Compare B-Tree and B+ Tree; show B-Tree insertion. *[7 Marks]*

Ans. A **B-Tree of order m** is a balanced m-way search tree. Each node has at most **m children and m − 1 keys**, non-root nodes have at least **⌈m/2⌉ children**, and **all leaves are at the same level**. In a **B+ Tree**, data is stored only in the leaves, and the leaves are **linked**.



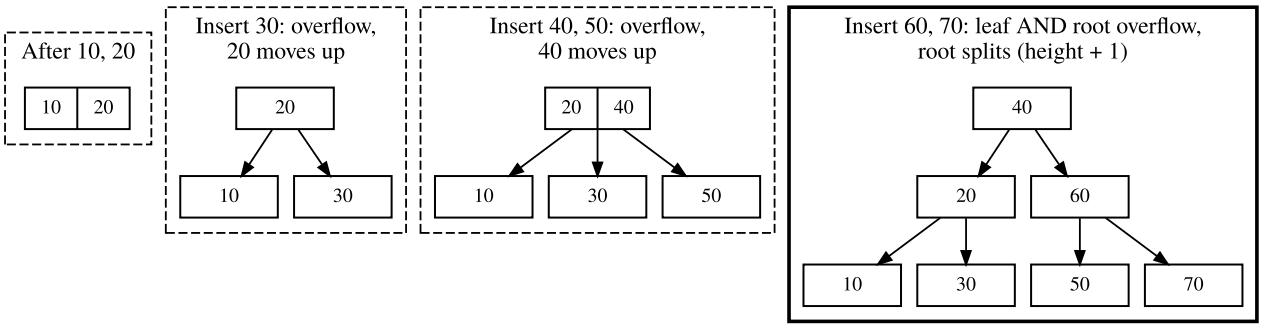
B-Tree of order 3: every node holds keys AND data pointers (*)



B+ Tree of order 3: data only in the leaves, and the leaves are linked in order

Insertion rule:

1. Insert into the correct leaf.
2. If it overflows, **split** it: the **median moves up**.
3. If the root splits, the height increases by 1.



B-Tree (order m = 3) insertion of 10, 20, ..., 70: each overflow pushes the median key up

Basis	B-Tree	B+ Tree
Data pointers	All nodes	**Leaves only**
Leaf links	No	**Yes** (sequential access)
Range queries	Slow	**Fast**
Search time	Variable	Uniform
Used in	File systems	**All DBMS indexes**

Conclusion: B+ Trees are the default index in **MySQL InnoDB, Oracle and PostgreSQL**.